Manoeuvre: Gesture-Based Intelligent Teaching and Presentation System

Project Report

Oct 22, 2026 · @Vedanth

1. Introduction

Manoeuvre is a wearable gesture glove paired with desktop software that lets a teacher move the cursor, change slides and annotate them without touching a keyboard or mouse.

1.1 Background and motivation

Lecturers constantly switch between speaking, operating presentation controls, moving a mouse and annotating slides. Each switch interrupts the flow of teaching and ties the presenter to the laptop. A wearable controller removes that friction: the presenter moves freely while hand motion and a few buttons drive the presentation.

The project is both a software engineering effort and a hardware system. A NodeMCU ESP8266 inside the glove reads an inertial measurement unit (IMU) and three buttons and sends structured events over Wi-Fi. A Windows desktop application, Manoeuvre Studio, turns those events into cursor movement, slide navigation and freehand annotation, and adds lecture management and AI-assisted teaching tools.

1.2 Problem statement

Traditional presentation workflows do not let a teacher control and annotate slides naturally while moving around a room. Building a gesture-based alternative raises several technical problems:

- IMU readings contain noise and drift, and relative hand motion must become usable cursor movement.
- Wi-Fi links can drop or change network addresses.
- Firmware, backend and frontend may use different event formats.
- Button presses and releases must stay synchronized.
- High-frequency movement events must be processed with low latency.

- An annotation must follow the hand's continuous path, not a straight line between its start and end points.
- A disconnect must never leave the cursor, a mouse button or an active stroke in an invalid state.

1.3 Objectives

Primary objectives:

1. Build a wearable glove for hands-free presentation interaction.
2. Enable cursor movement from IMU-derived motion.
3. Support physical-button-controlled annotation, mode switching and slide navigation.
4. Establish a persistent, recoverable device connection.
5. Render continuous freehand strokes from glove movement.
6. Integrate the glove with Manoeuvre Studio while preserving its existing lecture, annotation and analytics features.
7. Provide a simple Windows startup and device-connection workflow.

Secondary objectives: calibration and sensitivity controls, device and connection status display, diagnostic telemetry, undo/redo and annotation management, AI-assisted teaching utilities, session analytics, and extensibility for future sensors and gesture modes.

1.4 Scope and limitations

In scope: the ESP8266 firmware, the Wi-Fi/WebSocket event pipeline, the FastAPI backend, the React/Electron desktop application, SQLite persistence, and the continuous annotation engine, on a Windows laptop.

Limitations:

- The target platform is Windows desktop only.
- Power, battery and charging details of the glove are not specified in the project documents and are not covered here.
- The sensor identity is not fully settled: the hardware reported WHO_AM_I `0x70` (MPU6500-compatible), while an earlier software report assumed an MPU6050 at address `0x68`.

- Earlier test results (21 automated tests) were reported by a previous build and have not been re-run; this report separates simulated results from physical hardware verification.
- AI features depend on a configured provider; a configured interface does not prove a live external service.

2. Literature Survey / Existing System

Existing ways to control a presentation hands-free fall into a few families, and none combines glove-based sensing, continuous annotation and lecture management in one system.

2.1 Review of existing systems

Approach	How it works	Strengths	Weaknesses for teaching
Handheld presentation clickers	Radio or USB remote with next/previous buttons and often a laser pointer	Cheap, simple, reliable	Navigation only; no cursor-level annotation; occupies the hand
Air mice (IMU-based handheld devices)	Gyroscope/accelerometer moves the pointer, buttons click	Free cursor movement	Held in the hand; no annotation workflow or lecture tooling
Camera-based gesture recognition	Vision models track hands or body from a webcam or depth sensor [1]	No wearable needed	Depends on lighting, camera placement and line of sight; higher compute load and latency
Interactive whiteboards and pen displays	Touch or pen input on a dedicated surface [2]	Natural handwriting	Fixed hardware, cost, presenter tied to the board

Approach	How it works	Strengths	Weaknesses for teaching
Wearable IMU gloves in research	Inertial sensors on the hand for gesture or sign recognition [3]	Works without a camera	Mostly gesture-classification research, rarely integrated with presentation software

The table summarizes general categories; the cited works in the References section should be checked and extended with the papers or product documents your course expects.

2.2 Gaps identified and proposed solution

Gaps:

- Clickers and air mice handle navigation or pointing but do not produce continuous freehand annotation stored with the slide.
- Camera-based systems add environmental dependencies and a vision pipeline that a classroom may not control.
- Research gloves are rarely connected to a full application with presentations, analytics and persistence.
- Many hobby builds treat the sensor, firmware and desktop app as separate pieces, which leads to duplicated cursor control, inconsistent event formats and stuck mouse buttons.

Proposed solution: Manoeuvre integrates a wearable IMU-and-button glove with a desktop teaching application. It uses one authoritative event path over WebSockets [4], a canonical event protocol, a single owner for native pointer movement, and an annotation engine that retains intermediate points so strokes follow the hand. Lecture management, AI-assisted tools and analytics are kept as part of the same product.

3. Software Process Model and Project Planning

Manoeuvre follows an iterative Agile approach organized as short sprints, each ending in a testable increment.

3.1 SDLC model chosen

Model: Agile (Scrum-style sprints). Justification:

- Hardware and software evolve together. Firmware, protocol and UI each needed to change as testing exposed problems (for example, the ESP32 versus ESP8266 mismatch and the straight-line annotation defect).
- Requirements were refined as the system was integrated; Waterfall would have locked decisions before the glove could be tested.
- Each sprint produces something verifiable, such as a compiled firmware, a passing protocol test or a working simulator run.
- Risks are visible early because every sprint ends with a demo of the working path.

3.2 Project schedule (sprint plan)

The sprints follow the implementation priorities in the project specification. Dates are left for you to fill from your actual calendar.

Sprint	Focus	Main deliverable	Dates
1	Hardware compatibility	ESP8266 board target, pin map and IMU identity confirmed; firmware compiles	TBD
2	Communication	One device endpoint, canonical event schema, correct LAN addressing, reconnect behavior	TBD
3	Event routing	Motion, buttons, mode changes and slide navigation reach their handlers	TBD
4	Continuous annotation	Intermediate-point capture, incremental rendering, stroke finalization, diagnostics	TBD
5	Input ownership	Single cursor owner; no stuck mouse buttons on disconnect	TBD
6	User experience	Device status, reconnect control, calibration and diagnostics in the UI	TBD
7	Deployment	Windows launcher, setup and troubleshooting documentation	TBD
8	Verification	Automated tests, builds, simulator tests, physical hardware checklist	TBD

3.3 Risk analysis

ID	Risk	Likelihood / impact	Mitigation
R1	Earlier software targeted ESP32 while the hardware is NodeMCU ESP8266	High / High	Verify board configuration, GPIOs, drivers and compile for ESP8266
R2	Sensor identity mismatch (WHO_AM_I `0x70` versus an assumed MPU6050 at `0x68`)	Medium / High	Check I²C address, identity register and a compatible driver
R3	Multiple WebSocket paths (ports 8000 and 8765) with different formats	Medium / High	Choose one authoritative path; use an explicit adapter if the legacy bridge stays
R4	Two controllers move the same cursor	Medium / High	Assign one owner for native pointer movement
R5	Annotation loses intermediate points (straight-line strokes)	High / High	Instrument event rate and point count; run curved-path tests
R6	Laptop LAN address changes	High / Medium	Configurable endpoint; documented connection procedure
R7	Simulator success mistaken for physical success	Medium / Medium	Separate physical verification checklist

Likelihood and impact ratings are the author's judgment and can be adjusted to your own assessment.

3.4 Cost and effort estimation (optional)

Hardware cost (NodeMCU ESP8266, IMU module, three buttons, battery and wiring) and total effort in person-hours are not stated in the project documents. Enter your actual figures here before submission if your course requires this subsection. The software stack is open source, so no license cost applies.

4. Requirements Analysis

The system must turn physical hand motion and button presses into reliable cursor, navigation and annotation actions on a Windows laptop.

4.1 Functional requirements

ID	Requirement
FR-1	Read IMU data and button states on the glove and transmit them as structured events over Wi-Fi
FR-2	Move the cursor from IMU-derived motion with configurable sensitivity, dead zone and axis inversion
FR-3	Support MODE, ACTION and SYSTEM buttons with debounced press and release events
FR-4	Switch between interaction modes (CURSOR, ANNOTATE, HIGHLIGHT, VOICE, slide navigation) as verified on the current firmware
FR-5	Navigate presentations (next and previous slide) from glove input
FR-6	Draw continuous freehand annotations while ACTION is held in ANNOTATE mode
FR-7	Finalize and persist a stroke on ACTION release; support undo, redo, clear and brush settings
FR-8	Show device connection state and heartbeat in the application
FR-9	Reconnect Wi-Fi and WebSocket automatically after temporary failures
FR-10	End any active stroke and release held mouse buttons on disconnect
FR-11	Manage PDF presentations, lecture sessions, annotations and session analytics

ID	Requirement
FR-12	Provide AI-assisted tools (slide explanation, takeaways, quiz questions, analogies, recap) when a provider is configured
FR-13	Provide calibration, sensitivity settings and a hardware simulator

4.2 Non-functional requirements

- **Performance:** low input latency; high-frequency motion handled without lag.
- **Reliability:** reliable event ordering; safe recovery from communication failures.
- **Maintainability:** clear separation of responsibilities; small firmware modules; minimal unnecessary dependencies.
- **Security:** Electron context isolation on, no Node.js in the renderer, validated IPC, secrets masked and never logged or committed.
- **Portability:** no hardcoded machine-specific paths or IP addresses; configurable endpoints.
- **Usability:** simple startup, connection, testing and recovery workflow.
- **Integrity of reporting:** simulated and physically verified behavior are reported separately.

4.3 Software Requirements Specification (SRS) summary

- **Purpose:** gesture-based control and annotation of presentations for educators.
- **Product perspective:** a glove (ESP8266 firmware) plus a desktop product (FastAPI backend, React frontend, Electron shell, SQLite).
- **Users:** teachers and presenters; developers maintaining the system.
- **Interfaces:** glove to backend over WebSocket; backend to frontend over a desktop WebSocket and REST; backend to Windows input APIs; backend to SQLite.
- **Constraints:** Windows only; the glove needs the laptop's reachable LAN address (loopback `127.0.0.1` cannot be used from the glove).
- **Assumptions:** glove and laptop share a Wi-Fi network; the IMU is I²C-connected on D1/D2.

4.4 Hardware and software requirements

Hardware

Item	Detail
Microcontroller	NodeMCU ESP8266
IMU	GY-91 / MPU6500-compatible; I²C SDA D2 (GPIO4), SCL D1 (GPIO5)
Buttons	MODE D5 (GPIO14), ACTION D6 (GPIO12), SYSTEM D7 (GPIO13); active-low with `INPUT_PULLUP`
Host	Windows laptop with Wi-Fi
Power	Battery, regulator and charging details to be documented from the assembly

Software

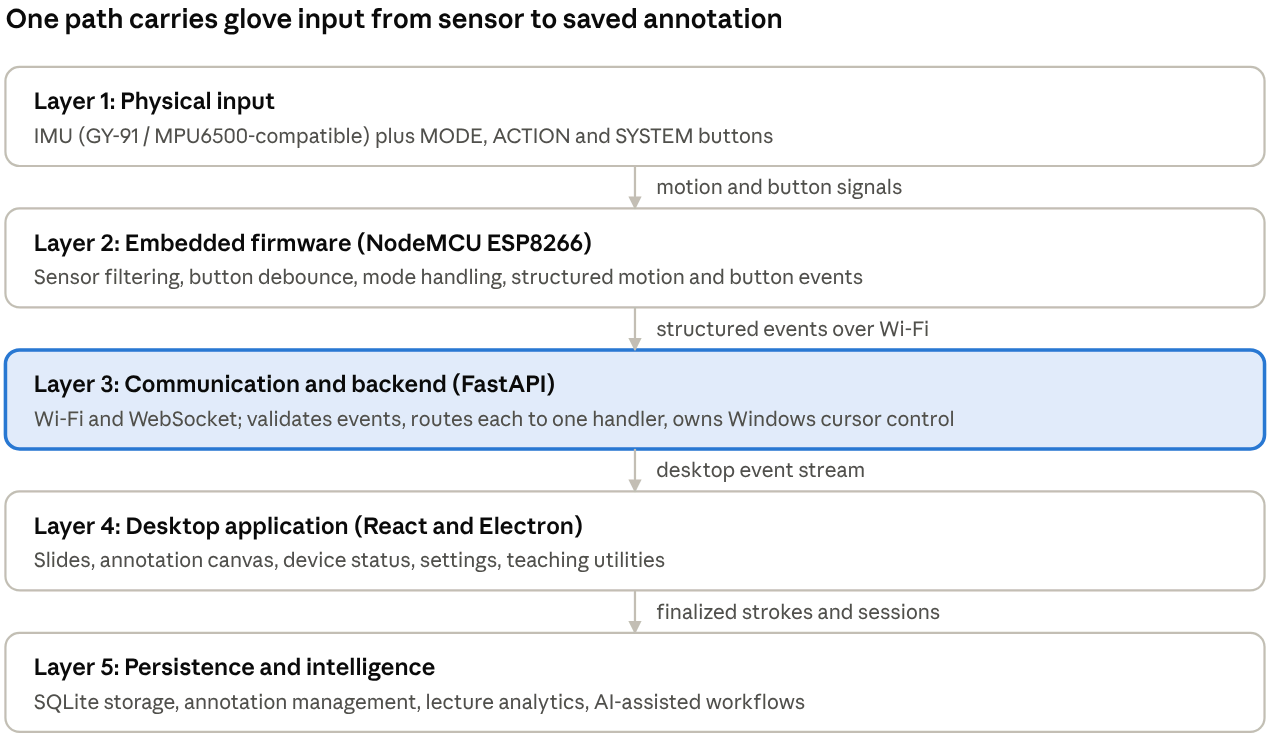
Layer	Technology
Firmware	Embedded C++ on the ESP8266 Arduino platform
Backend	Python, FastAPI, WebSockets
Frontend	React 18, TypeScript, Vite, Tailwind CSS
Desktop shell	Electron
Database	SQLite
Operating system	Windows

5. System Design

The design is a five-layer pipeline in which each input effect has exactly one authoritative owner.

5.1 System architecture

The five layers are physical input (IMU and buttons), embedded firmware, communication and backend, desktop application, and persistence with intelligence services. The target data flow is: glove sensor or button, ESP8266 firmware, Wi-Fi, WebSocket, backend event dispatcher, desktop event stream, application action.



system architecture · 5 layers

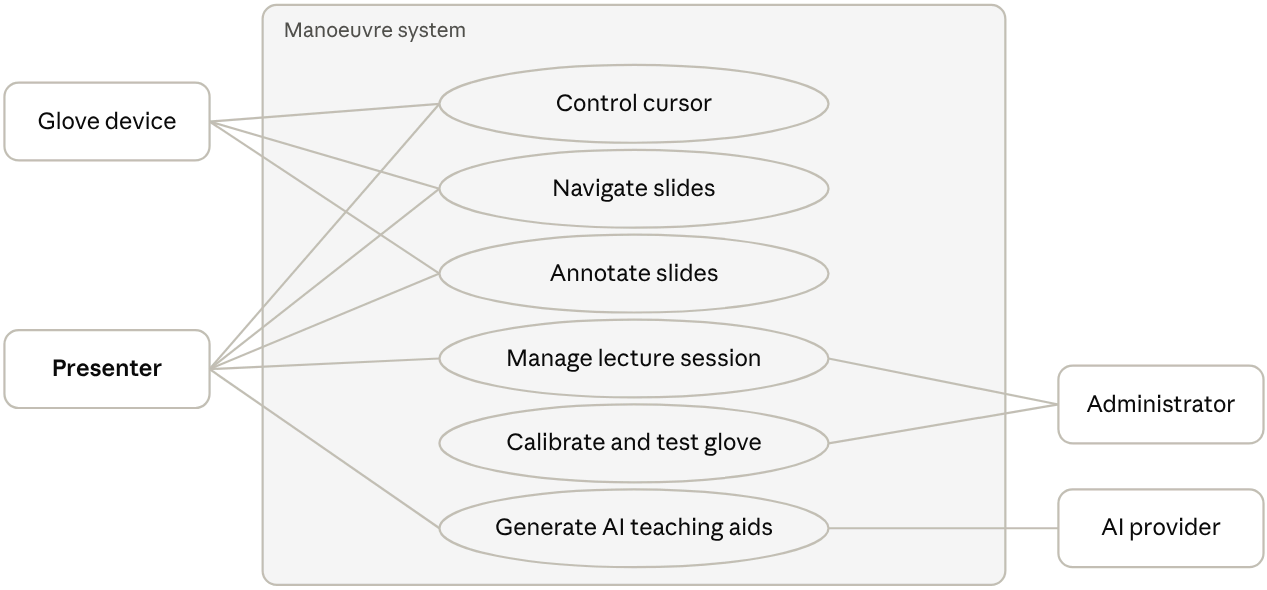
Endpoints reported by the earlier engineering report, to be confirmed against the repository: `ws://127.0.0.1:8000/ws/device` (hardware), `ws://127.0.0.1:8000/ws/desktop` (frontend stream), `ws://127.0.0.1:8765/ws` (legacy bridge) and Vite on port 5173. The glove must use the laptop's LAN address, not `127.0.0.1`.

5.2 Use case diagram

Actors: **Presenter** (wears the glove), **Glove device**, **Administrator/Developer** (configures and tests), **AI provider** (external service).

Actor	Use cases
Presenter	Move cursor; switch mode; navigate slides; annotate; undo/redo/clear; start lecture session; use AI quick prompts; review session recap
Glove device	Send motion events; send button press/release; send heartbeat; receive server commands
Administrator/Developer	Upload presentations; calibrate sensors; adjust sensitivity and dead zone; test buttons; run simulator; configure AI provider
AI provider	Generate explanations, takeaways, quiz questions, analogies and recaps

The presenter drives live control while the administrator configures



use case diagram · 4 actors, 6 use cases

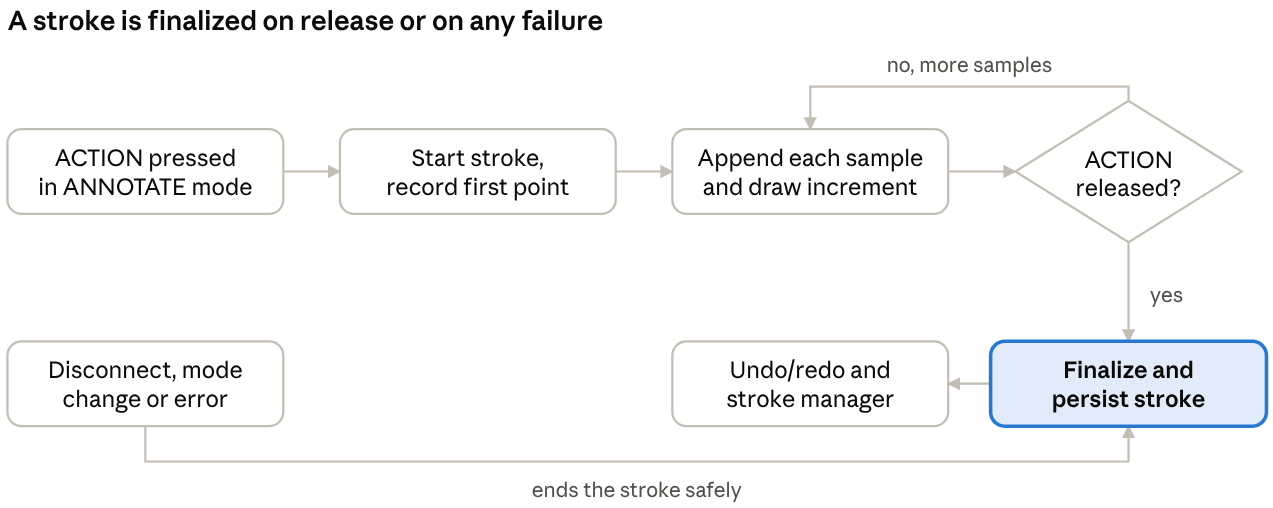
5.3 Class diagram

The project documents describe modules rather than a full class listing. The table gives the main components and their responsibilities; replace it with the actual classes from the repository if your course requires a formal UML class diagram (draw.io or StarUML).

Component	Layer	Key responsibilities	Collaborates with
SensorManager	Firmware	I²C init, IMU identity check, calibration, filtering, dead zone	GestureModeManager
ButtonManager	Firmware	GPIO setup, debounce, press/release events	GestureModeManager
GestureModeManager	Firmware	Holds current mode; interprets sensor and button events	NetworkManager
NetworkManager	Firmware	Wi-Fi and WebSocket connect/reconnect, heartbeat, event delivery	Backend device endpoint
Device dispatcher	Backend	Validates and routes events to one handler each	Cursor controller, presentation controller, desktop broadcast
Cursor controller	Backend	Windows pointer movement via `ctypes`; sensitivity, dead zone, held-button cleanup	Dispatcher
Presentation controller	Backend	Loads PDF slides, tracks state, changes slide	Dispatcher, frontend
Repository layer	Backend	SQLite access and migrations	All services
LectureWorkspaceScreen	Frontend	Slide display, annotation canvas, brush, undo/redo, zoom, AI prompts	Desktop WebSocket
Electron main/preload/overlay	Desktop shell	Window, typed IPC, backend process, transparent overlay	Frontend

5.4 Sequence / activity diagrams

The annotation lifecycle is the most important behavior, because a missed intermediate sample turns a curve into a straight line.



annotation lifecycle activity diagram

For presentation navigation, the sequence is: glove navigation input, firmware event, backend validation and dispatch, presentation controller, active slide changes, frontend updates.

5.5 ER diagram / database design

SQLite is used with write-ahead logging, foreign-key constraints, migrations and repository functions. The earlier report lists 12 tables, to be confirmed against the actual migrations.

Table	Purpose
`user_profiles`	User identity
`user_settings`	Preferences such as sensitivity, dead zone, theme, AI provider
`devices`	Registered glove devices
`presentations`	Uploaded presentations
`slides`	Slides belonging to a presentation

Table	Purpose
`lecture_sessions`	Lecture runs linked to a presentation
`annotations`	Strokes: slide link, coordinate sequence, color, width
`lecture_events`	Timeline of events during a session
`transcripts`	Lecture transcripts
`ai_interactions`	AI prompts and responses
`ai_summaries`	Generated summaries and recaps
`ai_questions`	Generated quiz questions

Main relationships: a presentation has many slides; a lecture session belongs to a presentation; annotations, lecture events, transcripts and AI records attach to a session or slide. Exact foreign keys should be read from the migrations before drawing the final ER diagram.

5.6 UI design / wireframes

The frontend (React 18, TypeScript, Vite, Tailwind CSS) has these screens:

- **Dashboard:** launch a lecture, device status, active presentation, AI readiness, recent sessions.
- **Device manager:** button testing, calibration, sensitivity and simulator controls; shows real connection and heartbeat state.
- **Presentation manager:** PDF upload, slide thumbnails, OCR layout inspection.
- **Lecture workspace:** slide display, freehand annotation, brush color and width, undo/redo, clear, zoom, quick AI prompts.
- **AI workspace:** explanations, takeaways, multiple-choice questions, analogies, code examples, recap.
- **Annotation manager:** per-slide stroke inspection and cleanup.
- **Teaching coach and analytics:** pacing, slide timing, transition timeline, searchable history, Markdown export.
- **Settings:** theme, cursor sensitivity, dead zone, AI provider, model names, masked API keys.

[Insert wireframe or screenshot of each screen here, for example from Figma or the running app.]

6. Implementation

The implementation splits into firmware, backend, frontend and desktop shell, joined by a single WebSocket event path.

6.1 Technology stack

Layer	Technology	Role
Firmware	Embedded C++ (ESP8266 Arduino platform)	Sensor reading, button debounce, modes, WebSocket client
Backend	Python, FastAPI, WebSockets	Device and desktop endpoints, event dispatch, REST, Windows input via `ctypes`
Frontend	React 18, TypeScript, Vite, Tailwind CSS	Lecture workspace, device manager, analytics, settings
Desktop shell	Electron	Window, preload bridge, backend process management, overlay
Database	SQLite (WAL, foreign keys, migrations)	Presentations, sessions, annotations, events, AI outputs

6.2 Module description

- **Sensor manager (firmware):** initializes I²C, checks the IMU identity, calibrates stationary offsets, filters readings and applies a dead zone. Raw angular velocity is never used directly as cursor displacement.
- **Button manager (firmware):** configures `INPUT_PULLUP` pins, debounces, and emits deterministic press and release events.
- **Gesture and mode manager (firmware):** keeps the current mode and interprets sensor and button events.
- **Network manager (firmware):** Wi-Fi and WebSocket reconnect, heartbeat and non-blocking event delivery, so button releases are never delayed by long waits.

- **Device dispatcher (backend):** validates each message (JSON, supported type, finite values, size) and routes it to exactly one handler.
- **Cursor controller (backend):** the only component that moves the native Windows pointer; releases held buttons on error or disconnect.
- **Presentation controller (backend):** loads PDF slides and changes the active slide on navigation events.
- **Lecture workspace (frontend):** shows slides, consumes the glove motion stream and renders strokes on a canvas.
- **Annotation engine (frontend):** keeps a mutable active stroke, appends every valid sample, draws incrementally, and commits on release.
- **Electron shell:** context isolation on, Node.js disabled in the renderer, typed IPC, backend process management and a transparent click-through overlay.

6.3 Key code snippets

The snippets below illustrate the design; replace them with the actual excerpts from the repository before submission.

Event envelope (Protocol v1, fields to be confirmed against the implementation):

```
{
  "protocol_version": 1,
  "type": "BUTTON",
  "event_id": 1042,
  "timestamp_ms": 183204,
  "payload": { "button": "ACTION", "state": "pressed" }
}
```

Button debounce (firmware, simplified):

```
const uint8_t ACTION_PIN = 12;           // D6 / GPIO12, active-low
const unsigned long DEBOUNCE_MS = 25;
bool stable = HIGH; bool lastRead = HIGH; unsigned long changedAt = 0;

void pollAction() {
  bool now = digitalRead(ACTION_PIN);
  if (now != lastRead) { lastRead = now; changedAt = millis(); }
  if (millis() - changedAt > DEBOUNCE_MS && now != stable) {
    stable = now;
    sendButtonEvent("ACTION", stable == LOW);  // press when LOW
  }
}
```

Incremental stroke capture (frontend, simplified):

```
const active = useRef<Point[] | null>(null);

function onMotion(p: Point) {
  if (!active.current) return;
  const prev = active.current.at(-1)!;
  active.current.push(p);                 // keep every intermediate sample
  ctx.beginPath(); ctx.moveTo(prev.x, prev.y);
  ctx.lineTo(p.x, p.y); ctx.stroke();     // draw only the new segment
}

function onActionRelease() {
  if (active.current) commitStroke(active.current); // persist, enable
undo/redo
  active.current = null;
}
```

Safe cleanup on disconnect (backend, simplified):

```
async def on_device_disconnect():
    cursor.release_all_buttons()      # no stuck mouse button
    await broadcast({"type": "STROKE_ABORT"})  # end any active stroke
```

7. Software Tools Used (Demo Screenshots)

Four tools support the project across development, design, testing and documentation. Confirm each one matches what you actually used, then add the screenshots where marked.

7.1 Tool 1: Git and GitHub (version control)

- **Purpose:** track every change to firmware, backend and frontend and share the code with faculty.
- **Usage in this project:** one repository holds the ESP8266 firmware, the FastAPI backend, the React/Electron app and the README with setup steps. Commits are made per sprint deliverable.
- **Screenshot to add:** the repository page showing folders and README; the commit history.
- **Caption:** *Figure 7.1: Manoeuvre repository structure and commit history on GitHub.*

7.2 Tool 2: Postman (API and WebSocket testing)

- **Purpose:** send test requests to the backend and inspect WebSocket messages without the glove.
- **Usage in this project:** call the health and REST endpoints, and connect to the device and desktop WebSocket endpoints to send sample events (for example a curved sequence of motion samples) and watch the responses.
- **Screenshot to add:** a health check response; a WebSocket session showing sent and received events.
- **Caption:** *Figure 7.2: Postman test of the backend health endpoint and device WebSocket.*

7.3 Tool 3: Arduino IDE or PlatformIO with the Serial Monitor (firmware)

- **Purpose:** compile, upload and debug the ESP8266 firmware.
- **Usage in this project:** select the NodeMCU ESP8266 board, compile, flash over USB and read Serial diagnostics: sensor initialization, Wi-Fi and WebSocket status, button transitions. Passwords are never printed.
- **Screenshot to add:** successful compile output; Serial Monitor showing sensor detection and a button press.

- **Caption:** *Figure 7.3: Firmware build and Serial Monitor diagnostics on the NodeMCU ESP8266.*

7.4 Additional tools and summary table

VS Code (or the Antigravity IDE used for the current build) is the code editor, and draw.io or StarUML can produce the formal UML versions of the diagrams in Section 5.

Tool	Purpose	Phase of SDLC	Usage in project
Git / GitHub	Version control and code sharing	Implementation, maintenance	Repository, commits per sprint, README, submission link
Postman	API and WebSocket testing	Testing	Backend REST and WebSocket checks, event replay
Arduino IDE / PlatformIO + Serial Monitor	Firmware build, upload, debugging	Implementation, testing	ESP8266 compile, flash, sensor and button diagnostics
VS Code / Antigravity	Code editing	Implementation	Python, TypeScript and C++ development
draw.io / StarUML	UML and ER diagrams	Design	Use case, class, sequence and ER diagrams
Figma (optional)	UI wireframes	Design	Screen layouts for the desktop application

8. Testing

Testing separates unit, integration and simulator tests from physical hardware verification, because a simulator cannot prove that the real IMU, buttons and Wi-Fi link work together.

8.1 Test plan and strategy

Level	Scope	Method
Firmware	Board target, compile, sensor init, debounce, reconnect	Compile for ESP8266; Serial Monitor checks on the device
Backend	Event schema, connect/disconnect, heartbeat timeout, routing, duplicate prevention, malformed and stale messages, shutdown cleanup	Automated tests (pytest-style) and Postman
Frontend	WebSocket handling, device state, mode changes, annotation lifecycle, undo/redo/clear, disconnect behavior	UI tests and manual checks
Continuous path	Curved input retained as intermediate points and rendered along the curve	Known curved coordinate sequence, automated and visual check
End to end	Movement, firmware, Wi-Fi/WebSocket, backend, desktop, cursor or annotation result	Simulator first, then the physical glove

The automated results reported by the earlier build (21 tests plus simulator and build checks) were not re-run for this report. Update the status column after running them again on the current build.

8.2 Test cases

Status key: **Reported** = passed in the earlier build report, not re-executed; **Pending** = to be run and recorded.

ID	Input	Expected output	Actual output	Status
TC-01	Backend REST and protocol tests	All pass	11 passed (earlier report)	Reported
TC-02	Database repository tests	All pass	5 passed (earlier report)	Reported
TC-03	WebSocket lifecycle test	Connect, exchange, close cleanly	1 passed (earlier report)	Reported

ID	Input	Expected output	Actual output	Status
TC-04	Frontend UI and navigation tests	All pass	4 passed (earlier report)	Reported
TC-05	Hardware simulator smoke test	Five sequences complete	5 sequences passed (earlier report)	Reported
TC-06	Frontend production build; Electron TypeScript compile	Both succeed	Passed (earlier report)	Reported
TC-07	Compile firmware for NodeMCU ESP8266	Build succeeds with correct board and pins	Fill in	Pending
TC-08	Power glove; read IMU	WHO_AM_I read and accepted by the driver	Fill in (reported `0x70`)	Pending
TC-09	Press and release ACTION once	One press and one release event, no bounce repeats	Fill in	Pending
TC-10	Glove joins Wi-Fi and the laptop's LAN endpoint	Device hello received; heartbeat shown in the app	Fill in	Pending
TC-11	Switch glove off, then on	Reconnect without restarting the app	Fill in	Pending
TC-12	Send malformed JSON and an unsupported event type	Rejected and logged; no crash	Fill in	Pending
TC-13	Move hand in CURSOR mode	Pointer follows; only one component moves it	Fill in	Pending
TC-14	Hold ACTION in ANNOTATE mode and draw a curve	Stroke has more than two points and follows the curve	Fill in	Pending
TC-15	Release ACTION	Stroke is committed and appears in undo/redo	Fill in	Pending

ID	Input	Expected output	Actual output	Status
TC-16	Disconnect the glove mid-stroke	Stroke ends safely; no held mouse button	Fill in	Pending
TC-17	SLIDE_NEXT and SLIDE_PREV events	Active slide changes once per event	Fill in	Pending

8.3 Test results and bug log

No new results are recorded yet. The table lists the integration defects identified in the project documents; confirm each against the current build.

Bug ID	Description	Suspected cause	Status
BUG-01	Annotation appears as a straight line from start to end	Missing intermediate samples: no pointer capture, excessive React updates, full-canvas redraws, a delay in the legacy PyAutoGUI bridge, or a canvas not consuming the glove stream	Open until the curved-path test passes
BUG-02	Firmware and software assume different boards	Earlier software report targeted ESP32 WROOM-32; hardware is NodeMCU ESP8266	Open until ESP8266 build verified
BUG-03	Sensor identity mismatch	WHO_AM_I `0x70` versus an assumed MPU6050 at `0x68`	Open until driver and address verified
BUG-04	Two WebSocket paths with different formats	Integrated backend (port 8000) and legacy bridge (port 8765)	Open until one path is chosen
BUG-05	Risk of duplicate pointer movement	Backend and bridge both moving the cursor	Open until single owner confirmed

9. Results and Discussion

The software path is specified and was reported as passing in simulation; physical results must still be recorded before the report is final.

9.1 Output screenshots of the working system

Add screenshots from the running system under each caption:

- *Figure 9.1: Dashboard showing glove connection state and heartbeat.*
- *Figure 9.2: Device manager with button test, calibration and sensitivity controls.*
- *Figure 9.3: Lecture workspace with a curved annotation drawn by the glove.*
- *Figure 9.4: Diagnostic view with event rate, active stroke point count and current mode.*
- *Figure 9.5: Annotation manager listing saved strokes per slide.*
- *Figure 9.6: Teaching coach with session recap and pacing analytics.*
- *Figure 9.7: Photo of the glove (NodeMCU, IMU, three buttons) and the Serial Monitor output.*

9.2 Analysis against objectives

Objective	Evidence available	Status
Wearable glove for hands-free interaction	Hardware design (ESP8266, IMU, three buttons)	Verify on physical device
IMU-driven cursor movement	Sensor pipeline specified (calibration, filtering, dead zone)	Verify TC-13
Button-controlled annotation, modes, navigation	Button and mode design; simulator sequences reported	Verify TC-09, TC-17
Persistent, recoverable connection	Heartbeat and reconnect design; WebSocket lifecycle test reported	Verify TC-10, TC-11
Continuous freehand strokes	Stroke lifecycle and diagnostics designed; defect BUG-01 identified	Verify TC-14, TC-15
Integration with Manoeuvre Studio	Existing screens and database retained	Verify no regression

Objective	Evidence available	Status
Simple Windows startup	Target startup sequence defined	Verify launcher and README

Discussion. The main lesson is that each part working alone is not enough: the pipeline must preserve intent from hand motion to saved stroke. The design therefore assigns one owner per input effect, keeps every intermediate sample, and cleans up on disconnect. The honest limits are that the earlier 21-test result comes from a previous build, the sensor identity needs confirming, and only physical testing can show that the IMU, buttons and Wi-Fi link behave together. Fill in measured latency, event rate and point counts here when you have them.

10. Conclusion and Future Enhancements

Manoeuvre is a unified gesture-driven teaching platform: a wearable glove, a single event path, a Windows desktop application and a continuous annotation engine designed to work together. Its core design decisions are ESP8266-specific firmware, one authoritative communication path, one owner for pointer movement, annotation strokes built from every intermediate sample, and safe cleanup when the connection fails. The final status depends on completing the current build and the physical tests in Section 8.

Future enhancements

- Add further sensors and gesture modes (for example recognized gestures beyond tilt and buttons).
- Add automatic device discovery so the laptop's changing LAN address needs no manual setting.
- Provide an installer and one-command startup for non-technical teachers.
- Extend analytics and AI features once a provider is confirmed live.
- Support macOS and Linux in addition to Windows.
- Document battery, regulator and charging design from the actual assembly.

11. References

[1] F. Zhang et al., "MediaPipe Hands: On-device real-time hand tracking," arXiv:2006.10214, 2020.

[2] *Add a source on interactive whiteboards or pen displays used in teaching (cited in Section 2.1).*

[3] *Add a research paper on wearable IMU gloves for gesture recognition (cited in Section 2.1).*

[4] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF RFC 6455, Dec. 2011.

[5] InvenSense, "MPU-6500 Product Specification," datasheet.

[6] Espressif Systems, "ESP8266EX Datasheet," datasheet.

[7] S. Ramírez, "FastAPI documentation," fastapi.tiangolo.com.

[8] OpenJS Foundation, "Electron documentation," electronjs.org.

References [1], [4] to [8] are well-known public sources, but check titles, versions and access dates before submission; [2] and [3] must be filled with sources you have read.

Appendix A: Project Links

- **GitHub repository:** *paste link here (public, with a README containing setup steps).*
- **Demo video:** *paste Google Drive link here, with access set to "Anyone with the link can view", and test it in an incognito window.*

Appendix B: Sample Code Listings

Insert longer listings here, for example the firmware main loop, the device dispatcher and the annotation canvas component. Keep full code on GitHub.

Submission Checklist

- [ ] GitHub repository is public or shared with the faculty, with a README and setup steps
- [ ] Drive video link opens in an incognito window without requesting access
- [ ] At least three tools are shown with screenshots, purpose and usage
- [ ] Sprint dates, cost and effort figures filled in (Section 3)
- [ ] Test cases TC-07 to TC-17 run and results recorded (Section 8)
- [ ] References [2] and [3] added; all references in IEEE format